UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos

(11[0|2]76)

Clase 01 • **Ejercicio 02** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 2: La Playlist de Música (TAD Lista Enlazada)

Contexto

En este ejercicio implementaremos una estructura de datos basada en objetos (Lista Enlazada Simple) sin utilizar las colecciones nativas de Java (`java.util`). El objetivo es comprender cómo construir un Tipo de Dato Abstracto (TAD) orientada a objetos que encapsule sus nodos internos y garantice la validez de su estado.

Enunciado

Una aplicación de reproducción de música requiere gestionar listas de reproducción personalizadas (*playlists*).

Cada **Canción** (`Cancion`) posee un título, un artista y una duración en segundos. La **Playlist** (`Playlist`) almacena las canciones de forma ordenada mediante una estructura interna de nodos enlazados (`NodoCancion`).

Tareas:

1. Diseñe la clase `Cancion` en Java. Asegúrese de que no se puedan crear canciones con una duración menor o igual a cero segundos.

2. Diseñe la clase `NodoCancion` que contenga una referencia a una `Cancion` y a un `NodoCancion` siguiente.
3. Diseñe la clase `Playlist` que encapsule la lista de nodos y ofrezca las siguientes operaciones públicas:
    - `agregarCancion(Cancion cancion)`: Agrega una canción al final de la lista de reproducción.
    - `reproducirSiguiente()`: Desencola y retorna la canción actual en reproducción, removiéndola de la lista.
    - `obtenerDuracionTotal()`: Retorna la suma total de la duración de todas las canciones en segundos.
    - `getCantidadCanciones()`: Retorna la cantidad actual de canciones en la lista de reproducción.
    - `estaVacia()`: Indica si la lista de reproducción no tiene canciones.

Restricciones y Tips

- **Prohibición de Colecciones:** Está estrictamente prohibido utilizar `java.util.List`, `ArrayList` o cualquier otra colección de la biblioteca estándar de Java. La estructura debe ser construida manualmente con nodos.
- **Encapsulamiento de Nodos:** La clase `NodoCancion` es un detalle de implementación interno. La interfaz pública de `Playlist` nunca debe recibir ni retornar objetos de tipo `NodoCancion`.
- **Manejo de Estados Inválidos:**
    - Si se intenta instanciar una `Cancion` con una duración menor o igual a cero, se debe lanzar `DuracionInvalidaException`.
    - Si se intenta llamar a `reproducirSiguiente()` sobre una playlist vacía, se debe lanzar `PlaylistVaciaException`.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Por qué es fundamental que la clase `Playlist` oculte completamente la existencia de la clase `NodoCancion` a los clientes que la utilizan?

2. ¿Qué ventajas de mantenibilidad y seguridad ofrece calcular o acumular la duración total dentro de `Playlist` en lugar de requerir que el cliente recorra la lista externamente para sumar los tiempos?